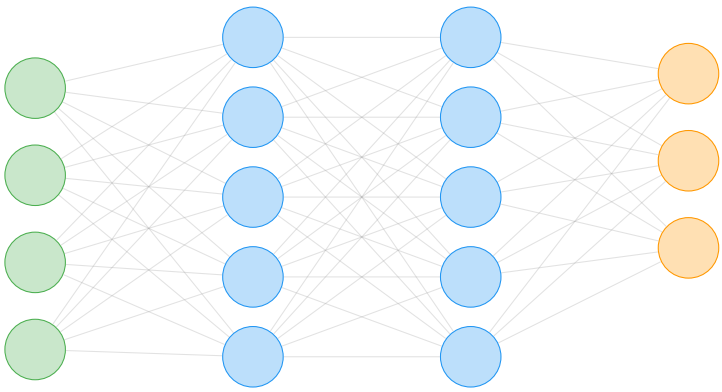
lstlistingname=Listing

Neural Networks: A Comprehensive Textbook for AI Students

A Comprehensive Guide for Artificial Intelligence Students



Textbook

G Qu. et al.

Supervised by:

Yunnan University
School of Intelligent Science and Technology
Kunming, China

March 2026

License

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.

About This Book

This comprehensive textbook on Neural Networks is designed for students and practitioners in Artificial Intelligence. It covers the fundamental concepts, mathematical foundations, and practical implementations of modern neural network architectures.

Main Features:

- Comprehensive coverage from basic perceptrons to advanced transformers
- Detailed mathematical derivations and explanations
- Practical examples using PyTorch framework
- GPU acceleration and CUDA programming concepts
- Real-world applications and case studies

Authors: G Qu. et al.

Supervisor:

Institution: Yunnan University, School of Intelligent Science and Technology

Year: 2026

Abstract

Neural networks have revolutionized the field of artificial intelligence, enabling breakthroughs in computer vision, natural language processing, speech recognition, and many other domains. This textbook provides a comprehensive introduction to neural networks, from fundamental concepts to state-of-the-art architectures.

The book is organized into eleven chapters, covering:

1. **Mathematical Foundations:** Linear algebra, calculus, and probability theory essentials for understanding neural networks.
2. **Activation Functions:** Comprehensive analysis of sigmoid, tanh, ReLU, and advanced activation functions with guidance on selection criteria.
3. **Loss Functions:** Detailed exploration of cross-entropy, mean squared error, and task-specific loss functions.
4. **Neural Network Architectures:** Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).
5. **Core Operations:** Convolution, pooling, and normalization operations with mathematical rigor.
6. **Advanced Mechanisms:** Skip connections and channel attention mechanisms including residual learning.
7. **Optimization:** Gradient descent algorithms, optimizers (SGD, Adam, AdamW), and learning rate schedulers.
8. **Classic Architectures:** From LeNet and AlexNet to ResNet, VGG, and the Transformer architecture.
9. **Computational Analysis:** Parameter counting and FLOPs computation for network efficiency analysis.
10. **Training and Frameworks:** Practical training procedures, hyperparameter tuning, and PyTorch implementation with CUDA concepts.
11. **Dataset Processing:** Data collection, annotation, augmentation, and splitting strategies.

Each chapter includes detailed mathematical derivations, practical examples, and references to seminal and recent research papers. Upon completing this textbook, readers will be equipped to design, implement, and train neural networks for various applications.

Keywords: Neural Networks, Deep Learning, Machine Learning, CNN, RNN, Transformer, Optimization, Backpropagation, PyTorch, Computer Vision, Natural Language Processing

Acknowledgements

We would like to express our sincere gratitude to all those who have contributed to the development of this textbook.

First and foremost, we thank the pioneering researchers whose groundbreaking work laid the foundation for modern neural networks, including Yann LeCun, Geoffrey Hinton, Yoshua Bengio, and many others whose contributions have shaped the field of deep learning.

We are grateful to our colleagues at Yunnan University for their valuable feedback and suggestions during the development of this material.

Special thanks to the open-source community, particularly the developers of PyTorch, NumPy, and other tools that have made deep learning accessible to researchers and practitioners worldwide.

Finally, we thank our families and friends for their patience and support throughout this project.

The Authors
March 2026

To all the curious minds who seek to understand
the mathematics of learning machines,
and to the future researchers who will push
the boundaries of artificial intelligence.

- Authors -

Contents

Table of contents 9

List of figures 11

List of tables 13

List of listings 13

1 Mathematical Foundations 1
- 1.1 Introduction . . . 1
- 1.2 Linear Algebra . . . 1
  - 1.2.1 Vectors and Matrices . . . 1
  - 1.2.2 Matrix Operations . . . 2
  - 1.2.3 Norms . . . 2
  - 1.2.4 Eigenvalues and Singular Values . . . 3
  - 1.2.5 Tensors . . . 3
- 1.3 Calculus . . . 4
  - 1.3.1 Derivatives and Gradients . . . 4
  - 1.3.2 Higher-Order Derivatives . . . 4
  - 1.3.3 Jacobian Matrix . . . 5
- 1.4 Probability Theory . . . 5
  - 1.4.1 Basic Probability . . . 5
  - 1.4.2 Common Distributions . . . 6
  - 1.4.3 Bayesian Probability . . . 6
- 1.5 Optimization . . . 7
  - 1.5.1 Optimization Problem . . . 7
  - 1.5.2 Gradient Descent . . . 7
  - 1.5.3 Challenges in Optimization . . . 7
- 1.6 Information Theory . . . 8
  - 1.6.1 Entropy . . . 8
  - 1.6.2 Cross-Entropy and KL Divergence . . . 8
- 1.7 Chapter Summary . . . 9
- 1.8 Further Reading . . . 9
- 1.9 Exercises . . . 9

2 Activation Functions 11
- 2.1 Introduction . . . 11
- 2.2 Neural Network Neurons . . . 11
- 2.3 Linear Activation Functions . . . 11
  - 2.3.1 Identity Function . . . 12
- 2.4 Sigmoid Family . . . 12
  - 2.4.1 Sigmoid (Logistic) . . . 12
  - 2.4.2 Hyperbolic Tangent (Tanh) . . . 13
- 2.5 Rectified Linear Unit (ReLU) Family . . . 13
  - 2.5.1 ReLU . . . 14
  - 2.5.2 Leaky ReLU . . . 15
  - 2.5.3 Parametric ReLU (PReLU) . . . 15
  - 2.5.4 Exponential Linear Unit (ELU) . . . 15

2.5.5 Swish / SiLU . . . 16
2.5.6 GELU (Gaussian Error Linear Unit) . . . 16
2.6 Special Activation Functions . . . 17
2.6.1 Softmax . . . 17
2.7 Choosing an Activation Function . . . 17
2.8 Practical Considerations . . . 18
2.8.1 Weight Initialization . . . 18
2.8.2 Batch Normalization with Activations . . . 18
2.9 Implementation in PyTorch . . . 18
2.10 Chapter Summary . . . 19
2.11 Further Reading . . . 19
2.12 Exercises . . . 20

3 Loss Functions **21**
3.1 Introduction . . . 21
3.2 The Role of Loss Functions . . . 21
3.3 Regression Loss Functions . . . 21
3.3.1 Mean Squared Error (MSE) . . . 21
3.3.2 Mean Absolute Error (MAE) . . . 22
3.3.3 Huber Loss . . . 22
3.4 Classification Loss Functions . . . 22
3.4.1 Binary Cross-Entropy . . . 22
3.4.2 Categorical Cross-Entropy . . . 23
3.5 Advanced Loss Functions . . . 23
3.5.1 Focal Loss . . . 23
3.5.2 Label Smoothing . . . 23
3.6 Specialized Loss Functions . . . 23
3.6.1 Triplet Loss . . . 23
3.6.2 Dice Loss . . . 23
3.7 Choosing a Loss Function . . . 24
3.8 Chapter Summary . . . 24
3.9 Further Reading . . . 24

4 Neural Network Architectures **25**
4.1 Introduction . . . 25
4.2 Deep Neural Networks (DNN) . . . 25
4.2.1 Architecture . . . 25
4.3 Convolutional Neural Networks (CNN) . . . 25
4.3.1 Convolutional Layer . . . 26
4.4 Recurrent Neural Networks (RNN) . . . 26
4.4.1 Architecture . . . 26
4.5 Long Short-Term Memory (LSTM) . . . 27
4.5.1 LSTM Gates . . . 27
4.6 Gated Recurrent Unit (GRU) . . . 28
4.6.1 GRU Equations . . . 28
4.7 Choosing Architecture . . . 28
4.8 Chapter Summary . . . 28

5 Core Operations in Neural Networks **29**
5.1 Introduction . . . 29
5.2 Convolution Operation . . . 29
5.2.1 2D Convolution . . . 29

5.2.2 Grouped and Depthwise Convolution . . . 29
5.3 Pooling Operations . . . 30
5.3.1 Max Pooling . . . 30
5.3.2 Average Pooling . . . 30
5.3.3 Global Pooling . . . 31
5.4 Normalization Techniques . . . 31
5.4.1 Batch Normalization . . . 31
5.4.2 Layer Normalization . . . 32
5.4.3 Instance Normalization . . . 32
5.5 Combining Operations . . . 32
5.5.1 Basic Conv Block . . . 32
5.6 Chapter Summary . . . 33

6 Skip Connections and Attention Mechanisms 35
6.1 Skip Connections . . . 35
6.1.1 Residual Connections (ResNet) . . . 35
6.1.2 Bottleneck Architecture . . . 35
6.1.3 Dense Connections (DenseNet) . . . 36
6.2 Attention Mechanisms . . . 36
6.2.1 Self-Attention . . . 36
6.2.2 Squeeze-and-Excitation (SE) Attention . . . 36
6.3 Chapter Summary . . . 37

7 Optimizers and Learning Rate Schedulers 39
7.1 Gradient Descent Optimization . . . 39
7.1.1 Batch Gradient Descent . . . 39
7.1.2 Stochastic Gradient Descent (SGD) . . . 39
7.1.3 Mini-Batch Gradient Descent . . . 39
7.2 Gradient Descent with Momentum . . . 39
7.3 Adaptive Learning Rate Methods . . . 40
7.3.1 AdaGrad . . . 40
7.3.2 RMSProp . . . 40
7.3.3 Adam (Adaptive Moment Estimation) . . . 41
7.3.4 AdamW (Adam with Weight Decay) . . . 41
7.4 Gradient Clipping . . . 41
7.5 Learning Rate Schedulers . . . 42
7.5.1 Step Decay . . . 42
7.5.2 Exponential Decay . . . 42
7.5.3 Cosine Annealing . . . 42
7.5.4 Warmup . . . 42
7.6 Optimizer Selection . . . 42
7.7 Chapter Summary . . . 42

8 Classic Network Architectures 45
8.1 Evolution of Neural Networks . . . 45
8.2 Perceptron and Early Neural Networks . . . 45
8.2.1 The Perceptron (1958) . . . 45
8.3 LeNet (1998) . . . 45
8.4 AlexNet (2012) . . . 45
8.5 VGGNet (2014) . . . 46
8.6 GoogLeNet / Inception (2014) . . . 47
8.7 ResNet (2015) . . . 47

8.8 DenseNet (2017) . . . 48
8.9 Transformer (2017) . . . 48
8.10 Chapter Summary . . . 49

9 Parameter and Computation Analysis **51**
9.1 Importance of Model Efficiency . . . 51
9.2 Parameter Counting . . . 51
9.2.1 Fully Connected Layer . . . 51
9.2.2 Convolutional Layer . . . 51
9.2.3 Batch Normalization . . . 52
9.3 FLOPs (Floating Point Operations) . . . 52
9.3.1 FLOPs for FC Layer . . . 52
9.3.2 FLOPs for Conv Layer . . . 52
9.4 Practical Examples . . . 53
9.4.1 LeNet-5 Parameter Count . . . 53
9.4.2 AlexNet Parameter Count . . . 53
9.5 Transfer Learning and Efficiency . . . 53
9.6 Chapter Summary . . . 54

10 Training Process and Frameworks **55**
10.1 The Training Loop . . . 55
10.2 Hyperparameters . . . 55
10.2.1 Learning Rate . . . 55
10.2.2 Batch Size . . . 56
10.3 Regularization . . . 56
10.3.1 Dropout . . . 56
10.3.2 Weight Decay . . . 57
10.4 Early Stopping . . . 57
10.5 Model Checkpointing . . . 57
10.6 Backpropagation . . . 57
10.7 PyTorch Framework . . . 58
10.8 CUDA and GPU Training . . . 58
10.9 Chapter Summary . . . 59

11 Dataset Processing **61**
11.1 Data Collection . . . 61
11.1.1 Data Sources . . . 61
11.2 Data Annotation . . . 61
11.2.1 Annotation Types . . . 61
11.2.2 Annotation Tools . . . 61
11.3 Data Splitting . . . 62
11.3.1 Basic Splits . . . 62
11.3.2 Stratified Splitting . . . 62
11.3.3 Cross-Validation . . . 62
11.4 Data Augmentation . . . 63
11.4.1 Image Augmentations . . . 63
11.4.2 Advanced Augmentation . . . 63
11.5 Data Loading . . . 63
11.6 Normalization . . . 64
11.6.1 Image Normalization . . . 64
11.7 Chapter Summary . . . 64

List of Figures

1.1 Gradient descent on a quadratic loss surface . . . 7
1.2 Schematic of loss landscape with local and global minima . . . 8

2.1 Singe neuron computation . . . 12
2.2 Sigmoid function and its derivative . . . 12
2.3 Tanh function and its derivative . . . 13
2.4 ReLU function and its derivative . . . 14
2.5 Comparison of ReLU variants . . . 16
2.6 Swish activation function . . . 16
2.7 Comparison of activation functions . . . 17

3.1 MSE vs MAE loss functions . . . 22

4.1 Deep Neural Network architecture . . . 25
4.2 Convolution operation . . . 26
4.3 RNN architecture . . . 27
4.4 LSTM cell structure . . . 27

5.1 Convolution with padding . . . 30
5.2 Max pooling 2×2, stride 2 . . . 31

6.1 Residual connection . . . 35
6.2 Self-attention mechanism . . . 36

7.1 Learning rate schedules . . . 42

11.1 Annotation types . . . 61
11.2 Data augmentation examples . . . 63

List of Tables

1.1 Common Probability Distributions in Deep Learning 6

2.1 Activation Function Selection Guide 18
2.2 Activation Function Quick Reference 19

3.1 Loss Function Selection Guide 24

4.1 Architecture Selection Guide 28

5.1 Normalization Comparison 32

7.1 Optimizer Selection Guide 43

8.1 Architecture Comparison 49

10.1 Hyperparameter Ranges 56

Chapter 1

Mathematical Foundations

1.1 Introduction

Neural networks are built upon a solid mathematical foundation that combines linear algebra, calculus, probability theory, and optimization. This chapter provides the essential mathematical prerequisites for understanding how neural networks learn and make predictions. We will cover these topics in the context of their applications in deep learning, providing both theoretical insights and practical intuitions.

The mathematics of neural networks can be viewed as the composition of linear transformations with nonlinear activation functions, optimized through gradient-based methods. Understanding this framework requires fluency in vector and matrix operations, differentiation of multivariate functions, and stochastic optimization.

1.2 Linear Algebra

Linear algebra provides the language for representing and manipulating data in neural networks. Neural networks fundamentally operate through linear transformations (matrix multiplications) combined with nonlinear activation functions.

1.2.1 Vectors and Matrices

A vector is an ordered collection of numbers that can represent a point in n-dimensional space. In neural networks, input data, hidden representations, and parameters are all represented as vectors or higher-dimensional tensors.

[Vector] A vector $\mathbf{x} \in \mathbb{R}^n$ is an ordered tuple of n real numbers:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = (x_1, x_2, \ldots, x_n)^\top \tag{1.1}$$

where x_i denotes the i-th component of the vector.

[Matrix] A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is a two-dimensional array of m rows and n columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \tag{1.2}$$

where a_{ij} denotes the element in the i-th row and j-th column.

[Neural Network Layer as Matrix Operation] A single fully-connected layer in a neu-

ral network performs the operation:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{1.3}$$

where:

- $\mathbf{x} \in \mathbb{R}^{d_{in}}$ is the input vector
- $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ is the weight matrix
- $\mathbf{b} \in \mathbb{R}^{d_{out}}$ is the bias vector
- $\mathbf{y} \in \mathbb{R}^{d_{out}}$ is the output vector

If $d_{in} = 784$ (MNIST image) and $d_{out} = 128$, then $\mathbf{W}$ contains $784 \times 128 = 100,352$ parameters.

1.2.2 Matrix Operations

- **Matrix Addition:** $\mathbf{C} = \mathbf{A} + \mathbf{B}$, where $c_{ij} = a_{ij} + b_{ij}$
- **Scalar Multiplication:** $c\mathbf{A}$, where every element is multiplied by c
- **Matrix Multiplication:** $\mathbf{C} = \mathbf{AB}$, where $c_{ij} = \sum_k a_{ik} b_{kj}$
- **Transpose:** $(\mathbf{A}^\top)_{ij} = a_{ji}$
- **Element-wise (Hadamard) Product:** $(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij} \cdot b_{ij}$

[Matrix Multiplication Properties] For matrices $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ of appropriate dimensions:

$$\mathbf{A}(\mathbf{BC}) = (\mathbf{AB})\mathbf{C} \quad \text{(associativity)} \tag{1.4}$$

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC} \quad \text{(distributivity)} \tag{1.5}$$

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top \quad \text{(transpose of product)} \tag{1.6}$$

Note: $\mathbf{AB} \neq \mathbf{BA}$ in general (non-commutativity).

1.2.3 Norms

Norms measure the "size" or "length" of vectors and matrices.

[Vector Norms] For a vector $\mathbf{x} \in \mathbb{R}^n$:

- L_1 norm (Manhattan norm): $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$
- L_2 norm (Euclidean norm): $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- L_p norm: $\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p\right)^{1/p}$
- L_∞ norm (max norm): $\|\mathbf{x}\|_\infty = \max_i |x_i|$

In neural networks, the L_2 norm is commonly used in regularization (weight decay), while the L_1 norm promotes sparsity.

[Weight Decay Regularization] Weight decay adds a penalty term to the loss function:

$$\mathcal{L}_{reg}(\mathbf{W}) = \mathcal{L}(\mathbf{W}) + \lambda \|\mathbf{W}\|_F^2 \tag{1.7}$$

where $\|\mathbf{W}\|_F = \sqrt{\sum_{i,j} w_{ij}^2}$ is the Frobenius norm (the L_2 norm of the flattened matrix).

1.2.4 Eigenvalues and Singular Values

Eigenvalues and singular values are fundamental for understanding linear transformations, dimensionality reduction, and network analysis.

[Eigenvalue and Eigenvector] For a square matrix $\mathbf{A} \in \mathbb{R}^{n\times n}$, if there exists a scalar λ and nonzero vector $\mathbf{v}$ such that:

$$\mathbf{A}\mathbf{v} = \lambda \mathbf{v} \tag{1.8}$$

then λ is called an eigenvalue and $\mathbf{v}$ is the corresponding eigenvector.

[Singular Value Decomposition] For any matrix $\mathbf{A} \in \mathbb{R}^{m\times n}$, the Singular Value Decomposition (SVD) is:

$$\mathbf{A} = \mathbf{U}^{\circ}\mathbf{V}^{\top} \tag{1.9}$$

where:

- $\mathbf{U} \in \mathbb{R}^{m\times m}$ and $\mathbf{V} \in \mathbb{R}^{n\times n}$ are orthogonal matrices
- $^{\circ} \in \mathbb{R}^{m\times n}$ is a diagonal matrix containing singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$

The singular values are related to eigenvalues by $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top\mathbf{A})}$.

SVD is crucial for Principal Component Analysis (PCA), which is used in understanding the dimensionality of neural network representations.

1.2.5 Tensors

Neural networks, especially convolutional neural networks, operate on higher-dimensional arrays called tensors.

[Tensor] A tensor is a generalization of vectors and matrices to higher dimensions:

- 0D tensor: scalar
- 1D tensor: vector
- 2D tensor: matrix
- 3D tensor: e.g., RGB images of shape (H, W, C)
- 4D tensor: batches of images of shape (N, H, W, C)
- 5D tensor: volumes of shape (D, H, W, C)

[Tensor Shapes in CNNs] For a batch of N color images of size $H \times W$ with 3 channels (RGB):

- Input tensor: $\mathbf{X} \in \mathbb{R}^{N\times H\times W\times 3}$
- After convolution with C filters: $\mathbf{Y} \in \mathbb{R}^{N\times H'\times W'\times C}$

PyTorch convention: (N, C, H, W) TensorFlow convention: (N, H, W, C)

1.3 Calculus

Calculus, particularly differential calculus, is essential for understanding how neural networks learn through gradient-based optimization.

1.3.1 Derivatives and Gradients

[Derivative] The derivative of a function $f : \mathbb{R} \to \mathbb{R}$ at point x is defined as:

$$f'(x) = \frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{1.10}$$

[Gradient] For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the gradient is a vector of partial derivatives:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \tag{1.11}$$

The gradient points in the direction of steepest ascent.

[Rules of Differentiation]

- Sum rule: $\frac{d}{dx}(f+g) = f' + g'$
- Product rule: $\frac{d}{dx}(fg) = f'g + fg'$
- Chain rule: $\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$
- Quotient rule: $\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f'g - fg'}{g^2}$

[Chain Rule in Neural Networks] For a two-layer network $f(\mathbf{x}) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$, the gradient computation uses the chain rule extensively. If we want $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1}$ for loss $\mathcal{L}$, we have:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{z}_1} \cdot \frac{\partial \mathbf{z}_1}{\partial \mathbf{W}_1} \tag{1.12}$$

where $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ and $\mathbf{z}_2 = \mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2$.

1.3.2 Higher-Order Derivatives

[Hessian Matrix] For a function $f : \mathbb{R}^n \to \mathbb{R}$, the Hessian matrix contains all second-order partial derivatives:

$$\mathbf{H} = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \tag{1.13}$$

The Hessian is symmetric if all second derivatives are continuous.

The Hessian is used in Newton's method and for understanding curvature of the loss landscape.

[Gradient Descent vs. Newton's Method]

- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla \mathcal{L}(\mathbf{w}_t)$
- Newton's method: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{H}^{-1} \nabla \mathcal{L}(\mathbf{w}_t)$

Newton's method uses second-order information (curvature) for potentially faster convergence, but computing the Hessian is expensive for large networks.

1.3.3 Jacobian Matrix

[Jacobian] For a vector-valued function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, the Jacobian is an $m \times n$ matrix of partial derivatives:

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \tag{1.14}$$

Each row i is the gradient of f_i.

The Jacobian generalizes the gradient to vector-valued functions, which is essential for backpropagation through complex computational graphs.

[Jacobian in Backpropagation] In backpropagation, we compute gradients of scalar outputs (like loss) with respect to vector inputs. If $\mathbf{y} = f(\mathbf{x})$ and L is a scalar loss, then:

$$\frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\top \frac{\partial L}{\partial \mathbf{y}} = \mathbf{J}^\top \nabla_{\mathbf{y}} L \tag{1.15}$$

The backward pass efficiently computes this product without explicitly forming the full Jacobian.

1.4 Probability Theory

Probability theory provides the framework for understanding uncertainty, making predictions, and designing loss functions.

1.4.1 Basic Probability

[Probability] For a random experiment with sample space Ω, a probability function P assigns to each event $A \subseteq \Omega$ a number $P(A)$ such that:

- $0 \leq P(A) \leq 1$ for all events A
- $P(\Omega) = 1$
- If $A_1, A_2, \ldots$ are mutually exclusive, then $P(\bigcup_i A_i) = \sum_i P(A_i)$

[Random Variable] A random variable X is a function that maps outcomes of a random experiment to numerical values:

- Discrete: $P(X = x)$ for countable values
- Continuous: described by probability density function $p(x)$

- **Expected Value:** $E[X] = \sum_x x \cdot P(X = x)$ or $\int x \cdot p(x)dx$
- **Variance:** $\text{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$
- **Standard Deviation:** $\sigma_X = \sqrt{\text{Var}(X)}$

1.4.2 Common Distributions

Table 1.1 – Common Probability Distributions in Deep Learning

Distribution	Formula	Applications
Bernoulli	$P(X = 1) = p, P(X = 0) = 1 - p$	Binary classification
Categorical	$P(X = k) = p_k$	Multi-class classification
Gaussian (Normal)	$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	Regression, weight initialization
Multinomial	Generalization of Bernoulli	Language modeling

[Softmax Distribution] The softmax function converts a vector of logits $\mathbf{z} \in \mathbb{R}^K$ into a probability distribution:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{1.16}$$

Properties:

- $\sum_{i=1}^{K} \text{softmax}(z_i) = 1$
- $\text{softmax}(z_i) \in (0, 1)$ for all i
- $\arg\max_i \text{softmax}(z_i) = \arg\max_i z_i$

Used for multi-class classification output.

1.4.3 Bayesian Probability

[Bayes' Rule]

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \tag{1.17}$$

In the context of machine learning:

$$P(\text{hypothesis}|\text{data}) = \frac{P(\text{data}|\text{hypothesis}) \cdot P(\text{hypothesis})}{P(\text{data})} \tag{1.18}$$

- $P(\text{hypothesis})$: Prior probability
- $P(\text{data}|\text{hypothesis})$: Likelihood
- $P(\text{hypothesis}|\text{data})$: Posterior probability

Bayesian methods are used in Bayesian neural networks for uncertainty estimation and in Bayesian optimization for hyperparameter tuning.

1.5 Optimization

Neural network training is fundamentally an optimization problem: finding parameters that minimize a loss function.

1.5.1 Optimization Problem

[Neural Network Training] Given:

- A neural network $f(\mathbf{x}; \theta)$ with parameters θ
- A dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$
- A loss function $\mathcal{L}(\theta)$

Find:

$$\theta^* = \arg\min_\theta \mathcal{L}(\theta) \tag{1.19}$$

where typically:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\mathbf{x}_i; \theta), \mathbf{y}_i) \tag{1.20}$$

1.5.2 Gradient Descent

[Batch Gradient Descent]

1. Initialize parameters $\theta^{(0)}$
2. For $t = 0, 1, 2, \ldots$:
   (a) Compute gradient: $\mathbf{g}_t = \nabla \mathcal{L}(\theta^{(t)})$
   (b) Update parameters: $\theta^{(t+1)} = \theta^{(t)} - \alpha \mathbf{g}_t$
3. Stop when convergence criteria are met

The learning rate $\alpha > 0$ controls the step size. Choosing an appropriate learning rate is crucial for successful training.

$$w_1 w_2 \mathcal{L}^2 + 0.5 * y^2 + 0.3 * x * y;$$

Figure 1.1 – Gradient descent on a quadratic loss surface

1.5.3 Challenges in Optimization

- **Local Minima:** The loss landscape is non-convex, containing many local minima. However, for large neural networks, local minima are often not a significant problem; saddle points are more common.
- **Vanishing/Exploding Gradients:** In deep networks, gradients can become very small (vanish) or very large (explode), making training difficult.
- **Plateaus:** Flat regions where gradients are near zero, leading to slow progress.
- **Loss Landscapes:** Neural network loss landscapes are complex, with many local minima, saddle points, and flat regions.

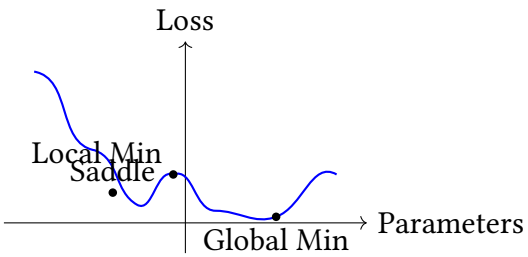


Figure 1.2 – Schematic of loss landscape with local and global minima

1.6 Information Theory

Information theory provides a framework for understanding loss functions, particularly cross-entropy.

1.6.1 Entropy

[Shannon Entropy] For a discrete random variable X with probability distribution $p(x)$:

$$H(X) = -\sum_x p(x) \log p(x) = E[-\log p(X)] \tag{1.21}$$

Entropy measures the "uncertainty" or "information content" of a distribution.

[Properties of Entropy]

- $H(X) \geq 0$ for all distributions
- $H(X) = 0$ if and only if $p(x) = 1$ for some x (deterministic)
- $H(X)$ is maximized for the uniform distribution
- Binary entropy: $H_b(p) = -p \log p - (1-p) \log(1-p)$

1.6.2 Cross-Entropy and KL Divergence

[Cross-Entropy] For two probability distributions p (true) and q (predicted):

$$H(p, q) = -\sum_x p(x) \log q(x) = H(p) + D_{KL}(p \| q) \tag{1.22}$$

[KL Divergence]

$$D_{KL}(p \| q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \tag{1.23}$$

KL divergence measures the "distance" from q to p (not a true metric, as it's asymmetric).

[KL Divergence Properties]

- $D_{KL}(p \| q) \geq 0$ (non-negativity)
- $D_{KL}(p \| q) = 0$ if and only if $p = q$
- $D_{KL}(p \| q) \neq D_{KL}(q \| p)$ (asymmetric)

[Cross-Entropy Loss] For classification with N samples and K classes:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik} \tag{1.24}$$

where y_{ik} is the ground truth (one-hot) and $\hat{y}_{ik} = \text{softmax}(z_{ik})$ is the prediction.

Minimizing cross-entropy is equivalent to minimizing the KL divergence between the true and predicted distributions, since $H(p)$ is constant for one-hot labels.

1.7 Chapter Summary

This chapter covered the essential mathematical foundations for neural networks:

1. **Linear Algebra:** Vectors, matrices, tensor operations, and norms are the building blocks for representing and manipulating data and parameters in neural networks.
2. **Calculus:** Derivatives, gradients, and the chain rule are fundamental for computing gradients during backpropagation.
3. **Probability Theory:** Distributions, expected values, and Bayes' rule provide the framework for understanding predictions and designing loss functions.
4. **Optimization:** Gradient descent and its variants are the workhorses of neural network training.
5. **Information Theory:** Entropy and KL divergence justify the use of cross-entropy loss.

These mathematical tools will be applied throughout the textbook as we explore activation functions, loss functions, network architectures, and training algorithms.

1.8 Further Reading

For deeper coverage of these topics:

- [1] - Chapter 2 covers mathematical foundations for deep learning
- [2] - Comprehensive treatment of linear algebra and probability
- **strang2009introduction** - Excellent introduction to linear algebra
- **cover2012elements** - Classic text on information theory

1.9 Exercises

1. **Linear Algebra:** Given a weight matrix $\mathbf{W} \in \mathbb{R}^{256\times512}$, how many parameters does this layer have if we include a bias term? Write the computational cost in FLOPs for a single forward pass.
2. **Calculus:** Compute the gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{W}}$ for a single-layer network with output $\mathbf{y} = \mathbf{W}\mathbf{x}$, where $\mathcal{L} = \|\mathbf{y} - \mathbf{t}\|^2$.

3. **Probability:** Show that the softmax function is invariant to additive constants: $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$ for any constant c.
4. **Optimization:** Implement gradient descent for a simple quadratic function $f(x) = x^2$ and analyze the effect of different learning rates.
5. **Information Theory:** Prove that $D_{KL}(p\|q) \geq 0$ using Jensen's inequality.

Chapter 2

Activation Functions

2.1 Introduction

Activation functions are mathematical functions applied to the output of each neuron (or layer) in a neural network. They introduce **nonlinearity** into the network, enabling it to learn complex patterns that linear functions cannot capture. Without activation functions, no matter how many layers a network has, it would simply perform linear transformations, equivalent to a single linear layer.

The choice of activation function significantly impacts:

- The network's ability to learn complex patterns
- Training stability and convergence speed
- Gradient flow during backpropagation
- The output range of neurons
- Computational efficiency

In this chapter, we will examine the mathematical properties, advantages, disadvantages, and practical applications of various activation functions.

2.2 Neural Network Neurons

Before diving into activation functions, let us understand their role in a neuron.

A single artificial neuron computes:

$$y = f(\mathbf{w}^\top \mathbf{x} + b) = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.1}$$

where:

- $\mathbf{x} \in \mathbb{R}^n$ is the input vector
- $\mathbf{w} \in \mathbb{R}^n$ is the weight vector
- $b \in \mathbb{R}$ is the bias
- $f(\cdot)$ is the activation function
- $y \in \mathbb{R}$ is the output

2.3 Linear Activation Functions

While not technically "activation functions" in the practical sense, linear functions are sometimes used in output layers.

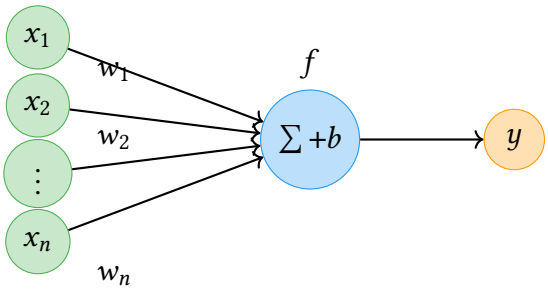


Figure 2.1 – A single neuron computes a weighted sum followed by an activation function

2.3.1 Identity Function

$$f(x) = x \tag{2.2}$$

- **Output range:** $(-\infty, \infty)$
- **Derivative:** $f'(x) = 1$
- **Use case:** Regression output layer
- **Not recommended:** Hidden layers (makes the network linear)

2.4 Sigmoid Family

The sigmoid family of activation functions squash their input into the range $(0, 1)$, making them useful for binary classification output layers.

2.4.1 Sigmoid (Logistic)

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \tag{2.3}$$

[Properties of Sigmoid]

- **Output range:** $(0, 1)$
- **Derivative:** $\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- **Monotonically increasing**
- **Symmetric around 0.5**
- **Smooth and differentiable everywhere**

$$x\sigma(x)\sigma(x)^2; Derivative\sigma'(x)$$

Figure 2.2 – The sigmoid function and its derivative

Smooth gradient

Outputs can be interpreted as probabilities

Historically widely used

Vanishing gradient: For large $|x|$, $\sigma'(x) \approx 0$

Not zero-centered: Outputs are always positive

Exponential computation: Slower than simpler functions

Saturation: Sigmoid saturates at 0 and 1

[Sigmoid in Binary Classification] For binary classification, the sigmoid output can be interpreted as:

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \tag{2.4}$$

The cross-entropy loss with sigmoid is:

$$\mathcal{L} = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \tag{2.5}$$

where $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$.

2.4.2 Hyperbolic Tangent (Tanh)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1 \tag{2.6}$$

[Properties of Tanh]

- **Output range:** $(-1, 1)$
- **Derivative:** $\tanh'(x) = 1 - \tanh^2(x)$
- **Zero-centered output**
- **Monotonically increasing**

$$x\tanh(x)\tanh(x)^2; Derivative\tanh'(x)$$

Figure 2.3 – The tanh function and its derivative

Zero-centered output (stronger gradients)

Stronger gradients than sigmoid

Outputs in $(-1, 1)$, often better for hidden layers

Vanishing gradient: Still present for large $|x|$

Exponential computation

Saturation at ± 1

2.5 Rectified Linear Unit (ReLU) Family

The ReLU family has become the dominant choice for hidden layers in modern deep learning due to their computational efficiency and effectiveness in training deep networks.

2.5.1 ReLU

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{2.7}$$

[Properties of ReLU]

- **Output range:** $[0, \infty)$
- **Derivative:**

$$\mathrm{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases} \tag{2.8}$$

- **Computationally efficient**
- **Sparse activation**

$$x(x)$$

Figure 2.4 – The ReLU function and its derivative

Very fast computation

Reduces vanishing gradient problem for positive activations

Sparse representations: Not all neurons activate simultaneously

Excellent empirical performance in deep networks

Biological plausibility

Dying ReLU problem: Neurons can "die" if they always output zero

Not zero-centered

Unbounded output

[Dying ReLU Problem] If a neuron's weights are updated such that $\mathbf{w}^\top \mathbf{x} + b \leq 0$ for all inputs, the neuron will output zero forever. This is called the "dying ReLU" or "dead neuron" problem. The neuron stops learning because the gradient is zero when the input is negative.

Solutions include:

- Leaky ReLU
- PReLU
- ELU
- Proper weight initialization
- Lower learning rates

2.5.2 Leaky ReLU

$$\text{LeakyReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} = \max(0, x) + \alpha \min(0, x) \tag{2.9}$$

where α is a small positive constant (typically $\alpha = 0.01$).

[Properties of Leaky ReLU]

- **Output range:** $(-\infty, \infty)$
- **Derivative:**

$$\text{LeakyReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{if } x \leq 0 \end{cases} \tag{2.10}$$

- **Small negative values allowed**

No dying ReLU problem

Allows small negative gradients

Approximately zero-centered

Hyperparameter α needs tuning

Performance may not be significantly better than ReLU

2.5.3 Parametric ReLU (PReLU)

$$\text{PReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{if } x \leq 0 \end{cases} \tag{2.11}$$

where α is a learnable parameter (can be per channel).

PReLU allows the network to learn the optimal slope for negative values, potentially improving performance.

2.5.4 Exponential Linear Unit (ELU)

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases} \tag{2.12}$$

where $\alpha > 0$ is a hyperparameter.

[Properties of ELU]

- **Output range:** $(-\alpha, \infty)$
- **Smooth for** $x < 0$
- **Negative values bring mean closer to zero**

No dying ReLU problem

x

Figure 2.5 – Comparison of ReLU and its variants

Outputs closer to zero mean

Smoother gradients

Exponential computation for negative values

Slower forward pass

2.5.5 Swish / SiLU

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{2.13}$$

Proposed by Ramachandran et al. (2017), Swish often outperforms ReLU in deeper networks.

[Properties of Swish]

- **Output range:** Approximately $(-0.31, \infty)$
- **Self-gated**
- **Non-monotonic** (dip for small negative values)
- **Derivative:** $\text{Swish}'(x) = \sigma(x) + x \cdot \sigma(x)(1 - \sigma(x)) = \text{Swish}(x) + \sigma(x)(1 - \text{Swish}(x))$

$x(x)$

Figure 2.6 – Swish vs ReLU

2.5.6 GELU (Gaussian Error Linear Unit)

$$\text{GELU}(x) = x \cdot \Phi(x) \tag{2.14}$$

where $\Phi(x)$ is the CDF of the standard normal distribution:

$$\Phi(x) = \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right] \tag{2.15}$$

A computationally efficient approximation:

$$\text{GELU}(x) \approx \frac{x}{2} \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right) \tag{2.16}$$

GELU is used in BERT, GPT, and many transformer architectures.

[Properties of GELU]

- **Stochastic regularizer** when used with dropout
- **Nonlinear in all regions**

- **Approximately equal to:** $0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$
- **Zero-centered output**

State-of-the-art performance in transformers

Combines benefits of ReLU and dropout

Smooth and non-monotonic

2.6 Special Activation Functions

2.6.1 Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}} \quad \text{for } i = 1, \ldots, K \tag{2.17}$$

Softmax is used for multi-class classification output layers.

[Properties of Softmax]

- **Output range:** Each output in $(0, 1)$
- **Sum constraint:** $\sum_i \text{softmax}(x_i) = 1$
- **Probabilistic interpretation**
- **Shift invariant:** $\text{softmax}(x + c) = \text{softmax}(x)$ for any constant c

Outputs are valid probability distributions

Enables cross-entropy loss optimization

Handles multi-class classification naturally

Only used in output layers (not hidden layers)

Can saturate when logits are very large

[Softmax in Practice] For numerical stability, implement softmax as:

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}} \tag{2.18}$$

This subtracts the maximum value to prevent overflow in the exponential.

2.7 Choosing an Activation Function

The choice of activation function depends on the specific task and network architecture. Here are general guidelines:

x

Figure 2.7 – Comparison of various activation functions

Table 2.1 – Activation Function Selection Guide

Layer Type	Recommended Functions	Notes
Hidden (General)	ReLU, GELU	ReLU is the default choice
Hidden (Deep)	ReLU, GELU, ELU	Avoid sigmoid/tanh in deep networks
Hidden (CNN)	ReLU, GELU	LeakyReLU for medical imaging
Hidden (RNN)	Tanh	Often default for RNNs
Hidden (Transformer)	GELU	Used in BERT, GPT, etc.
Output (Binary)	Sigmoid	For probability output
Output (Multi-class)	Softmax	For probability distribution
Output (Regression)	Identity / None	For unbounded output
Output (Bounded)	Sigmoid, Tanh	For $[0, 1]$ or $[-1, 1]$ output

2.8 Practical Considerations

2.8.1 Weight Initialization

Different activation functions work best with appropriate weight initialization:

- **Sigmoid/Tanh:** Xavier/Glorot initialization ($\mathcal{N}(0, \sqrt{2/(n_{in} + n_{out})})$)
- **ReLU (and variants):** He initialization ($\mathcal{N}(0, \sqrt{2/n_{in}})$)
- **GELU:** He initialization works well

See Chapter 7 on Optimization for more details.

2.8.2 Batch Normalization with Activations

When using batch normalization, the activation function typically comes **after** batch norm:

$$\mathbf{y} = f(\mathrm{BN}(\mathbf{W}\mathbf{x} + \mathbf{b})) \tag{2.19}$$

This ordering was popularized by the original BatchNorm paper and is now standard practice.

2.9 Implementation in PyTorch

– Activation Functions in PyTorch

python import torch import torch.nn as nn

Common activations relu = nn.ReLU() leaky$_r elu$ = $nn.LeakyReLU(0.01) slope for negative values prelu = nn.PReLU() learnable slope elu =$

$nn.ELU(alpha = 1.0)selu = nn.SELU()Self - NormalizingNeuralNetworksgelu = nn.GELU()UsedinTransformerssilu = nn.SiLU()SameasSwishsigmoid = nn.Sigmoid()tanh = nn.Tanh()softmax = nn.Softmax(dim = 1)$
Example usage x = torch.randn(32, 128) batch of 32, 128 features y = relu(x) Apply ReLU Softmax with dimension specification logits = torch.randn(32, 10) 10 classes probs = softmax(dim=1)(logits)
GELU in transformers transformer$_layer = nn.TransformerEncoderLayer(d_model = 512, nhead = 8, activation =' gelu'orusenn.GELU()directly)$

2.10 Chapter Summary

1. **Activation functions introduce nonlinearity** into neural networks, enabling them to learn complex patterns.
2. **Sigmoid and Tanh** squash inputs to bounded ranges but suffer from vanishing gradients.
3. **ReLU** is computationally efficient and reduces vanishing gradients but can suffer from dying neurons.
4. **Variants like Leaky ReLU, PReLU, and ELU** address the dying ReLU problem.
5. **GELU** has become popular in transformer architectures.
6. **Softmax** is used for multi-class classification output layers.
7. **Choice depends on** task, architecture, and training stability.

Table 2.2 – Activation Function Quick Reference

Function	Formula	Range	Common Use
Sigmoid	$1/(1+e^{-x})$	$(0,1)$	Binary output
Tanh	$(e^x - e^{-x})/(e^x+e^{-x})$	$(-1,1)$	RNN, rarely CNN
ReLU	$\max(0,x)$	$[0,\infty)$	Default hidden
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty,\infty)$	Prevent dying
ELU	x if $x>0$, else $\alpha(e^x-1)$	$(-\alpha,\infty)$	Better gradients
GELU	$x\cdot\Phi(x)$	varies	Transformers
Softmax	$e^{x_i}/\sum e^{x_j}$	$(0,1)^K$	Multi-class output

2.11 Further Reading

- Ramachandran et al., "Searching for Activation Functions" (2017) - Systematic comparison of activation functions
- Hendrycks and Gimpel, "Gaussian Error Linear Units (GELU)" (2016) - Introduction of GELU

- Clevert et al., "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)" (2015)
- Goodfellow et al., "Deep Learning" Chapter 6

2.12 Exercises

1. **Gradient Analysis:** Compute the gradient of the ELU function for both positive and negative inputs.
2. **Comparison:** Plot the activation functions covered in this chapter and their derivatives using Python/Matplotlib.
3. **Dying ReLU:** Implement a simple experiment to demonstrate the dying ReLU problem and show how LeakyReLU addresses it.
4. **Numerical Stability:** Show why subtracting $\max(x)$ from logits before computing softmax improves numerical stability.
5. **Custom Activation:** Design and implement a custom activation function that combines properties of ReLU and GELU. Test it on a simple image classification task.
6. **Self-Normalization:** Research SELU (Scaled Exponential Linear Unit) and explain how it enables self-normalizing neural networks.

Chapter 3

Loss Functions

3.1 Introduction

Loss functions (also called objective functions, cost functions, or error functions) are the mathematical measures that quantify how well a neural network's predictions match the true targets. The training process aims to minimize this loss by adjusting the network's parameters through gradient-based optimization.

The choice of loss function depends on:

- The type of task (classification, regression, etc.)
- The distribution of the target variable
- The desired properties of the solution (sparsity, robustness, etc.)
- Computational considerations

3.2 The Role of Loss Functions

Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ and a model $f(\mathbf{x}; \theta)$, the empirical risk is:

$$\mathcal{R}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\mathbf{x}_i; \theta), \mathbf{y}_i) \tag{3.1}$$

where $\ell(\cdot, \cdot)$ is the loss function.

Training seeks to find:

$$\theta^* = \arg\min_{\theta} \mathcal{R}(\theta) \tag{3.2}$$

The loss function should:

1. Be differentiable (for gradient-based optimization)
2. Penalize larger errors appropriately
3. Encourage correct behavior for the specific task
4. Have useful statistical interpretations

3.3 Regression Loss Functions

Regression tasks predict continuous values.

3.3.1 Mean Squared Error (MSE)

The Mean Squared Error (L2 loss):

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|^2 \tag{3.3}$$

[Properties of MSE]

- **Gradient:** $\frac{\partial \ell}{\partial \hat{y}} = -2(\mathbf{y} - \hat{\mathbf{y}})$
- **Convex for linear models**
- **Sensitive to outliers**

$$e = y - \hat{y}^2; {}^2|e|$$

Figure 3.1 – MSE vs MAE loss functions

3.3.2 Mean Absolute Error (MAE)

The Mean Absolute Error (L1 loss):

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^{N} \|\mathbf{y}_i - \hat{\mathbf{y}}_i\|_1 \tag{3.4}$$

Robust to outliers but non-differentiable at zero.

3.3.3 Huber Loss

Combines MSE and MAE:

$$\mathcal{L}_{\text{Huber}}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta \cdot |y - \hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases} \tag{3.5}$$

3.4 Classification Loss Functions

Classification tasks predict discrete class labels.

3.4.1 Binary Cross-Entropy

For binary classification ($y \in \{0, 1\}$):

$$\mathcal{L}_{BCE} = -\frac{1}{N} \sum_{i=1}^{N} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] \tag{3.6}$$

where $\hat{y}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$.

Probabilistic interpretation

Equivalent to MLE for binary classification

Strong gradients even for confident wrong predictions

3.4.2 Categorical Cross-Entropy

For multi-class classification:

$$\mathcal{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log\hat{y}_{ik} \tag{3.7}$$

where $\hat{y}_{ik} = \text{softmax}(z_{ik})$.

A key property is the gradient:

$$\frac{\partial\mathcal{L}}{\partial z_k} = \hat{y}_k - y_k \tag{3.8}$$

3.5 Advanced Loss Functions

3.5.1 Focal Loss

Addresses class imbalance in object detection:

$$\mathcal{L}_{\text{Focal}} = -\alpha_t(1-\hat{y}_t)^{\gamma}\log(\hat{y}_t) \tag{3.9}$$

where $\gamma \geq 0$ focuses training on hard examples.

3.5.2 Label Smoothing

Regularizes by softening hard labels:

$$y_{LS} = y(1-\epsilon) + \frac{\epsilon}{K} \tag{3.10}$$

where $\epsilon \in [0, 1]$ is the smoothing parameter.

3.6 Specialized Loss Functions

3.6.1 Triplet Loss

For metric learning:

$$\mathcal{L}_{\text{Triplet}} = \max(0, \|f_a - f_p\|^2 - \|f_a - f_n\|^2 + \alpha) \tag{3.11}$$

Ensures positive pairs are closer than negative pairs by margin α.

3.6.2 Dice Loss

For segmentation tasks:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2|A\cap B|}{|A| + |B|} \tag{3.12}$$

Handles class imbalance in segmentation.

3.7 Choosing a Loss Function

Table 3.1 – Loss Function Selection Guide

Task	Loss	Notes
Binary Classification	BCE	Standard choice
Multi-class	Cross-Entropy	With label smoothing
Regression	MSE	Sensitive to outliers
Robust Regression	MAE/Huber	Outlier resistant
Imbalanced Classification	Focal Loss	Focus on hard examples
Segmentation	Dice + CE	Handle class imbalance

3.8 Chapter Summary

Loss functions are fundamental to neural network training:

1. MSE is standard for regression but sensitive to outliers
2. Cross-entropy is standard for classification
3. Advanced losses address specific challenges like class imbalance
4. Label smoothing provides regularization
5. Task-specific losses exist for segmentation and metric learning

3.9 Further Reading

- Goodfellow et al., "Deep Learning" Chapter 6
- Lin et al., "Focal Loss for Dense Object Detection" (ICCV 2017)
- Milletari et al., "V-Net" (MICCAI 2016)

Chapter 4

Neural Network Architectures

4.1 Introduction

Neural network architectures define how neurons are organized and connected to process information. Different architectures are suited for different types of data and tasks. This chapter covers the three fundamental neural network architectures: Deep Neural Networks (DNN), Convolutional Neural Networks (CNN), and Recurrent Neural Networks (RNN).

4.2 Deep Neural Networks (DNN)

A Deep Neural Network consists of multiple layers of neurons, where information flows from input to output without loops.

4.2.1 Architecture

A fully-connected DNN performs:

$$\mathbf{h}^{(l)} = f^{(l)}(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \tag{4.1}$$

where $l = 1, \ldots, L$ indexes the layers.

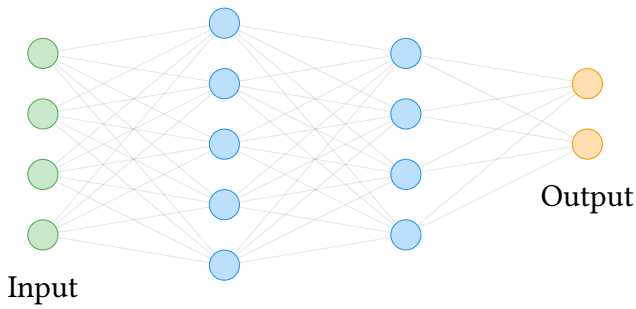


Figure 4.1 – Deep Neural Network architecture

[DNN Characteristics]

- **Parameter count:** $\sum_{l=1}^{L}(n^{(l-1)} \cdot n^{(l)} + n^{(l)})$
- **Universal approximation:** Can approximate any continuous function with sufficient width/depth
- **Suitable for:** Tabular data, feature-based classification

4.3 Convolutional Neural Networks (CNN)

CNNs exploit spatial structure in data through local connections and parameter sharing.

4.3.1 Convolutional Layer

The convolution operation:

$$y[i, j] = \sum_m \sum_n w[m, n] \cdot x[i + m, j + n] \tag{4.2}$$

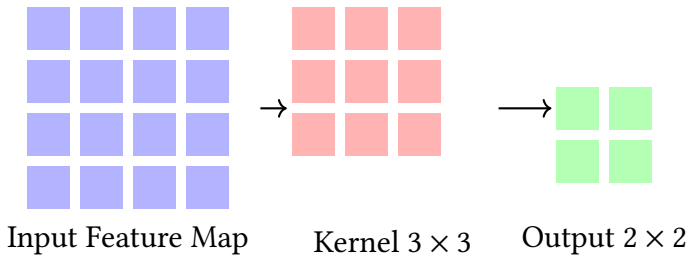


Figure 4.2 – Convolution operation

Key parameters:

- **Kernel size:** $K \times K$ (typically 3×3, 5×5, 7×7)
- **Stride:** s (step size, typically 1 or 2)
- **Padding:** p (zero-padding to control output size)

Output size:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - K}{s} + 1 \right\rfloor \tag{4.3}$$

[CNN Advantages]

- **Local receptive field:** Each neuron sees only a local patch
- **Parameter sharing:** Same weights across spatial locations
- **Translation equivariance:** Shift in input $\longrightarrow$ Shift in output
- **Suitable for:** Images, video, spatial data

4.4 Recurrent Neural Networks (RNN)

RNNs process sequential data by maintaining hidden state.

4.4.1 Architecture

For input sequence $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(T)}$:

$$\mathbf{h}^{(t)} = f(\mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{b}_h) \tag{4.4}$$

$$\mathbf{y}^{(t)} = g(\mathbf{W}_{hy}\mathbf{h}^{(t)} + \mathbf{b}_y) \tag{4.5}$$

[RNN Characteristics]

- **Sequential processing:** Can handle variable-length sequences

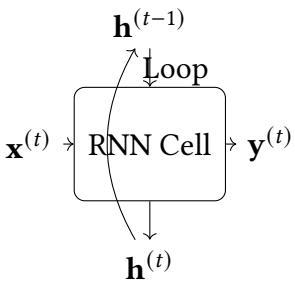


Figure 4.3 – RNN architecture

- **Shared parameters:** Same weights across time steps
- **Memory:** Hidden state captures past information
- **Suitable for:** Text, time series, audio, video

[Vanishing Gradient Problem] Standard RNNs suffer from vanishing gradients during backpropagation through time (BPTT), making it difficult to learn long-range dependencies. This led to the development of LSTM and GRU architectures.

4.5 Long Short-Term Memory (LSTM)

LSTM addresses the vanishing gradient problem with gating mechanisms.

4.5.1 LSTM Gates

$$\mathbf{f}^{(t)} = \sigma(\mathbf{W}_f[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_f) \quad \text{(forget gate)} \tag{4.6}$$

$$\mathbf{i}^{(t)} = \sigma(\mathbf{W}_i[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_i) \quad \text{(input gate)} \tag{4.7}$$

$$\mathbf{o}^{(t)} = \sigma(\mathbf{W}_o[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_o) \quad \text{(output gate)} \tag{4.8}$$

$$\tilde{\mathbf{c}}^{(t)} = \tanh(\mathbf{W}_c[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_c) \quad \text{(cell candidate)} \tag{4.9}$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)} \quad \text{(cell state)} \tag{4.10}$$

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)}) \quad \text{(hidden state)} \tag{4.11}$$

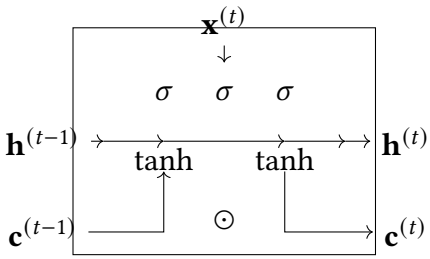


Figure 4.4 – LSTM cell structure

[LSTM Advantages]

- **Selective memory:** Gates control what to remember/forget

- **Long-range dependencies:** Cell state enables gradient flow
- **Widely used:** NLP, speech, time series

4.6 Gated Recurrent Unit (GRU)

GRU simplifies LSTM with fewer gates while maintaining performance.

4.6.1 GRU Equations

$$\mathbf{z}^{(t)} = \sigma(\mathbf{W}_z[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(update gate)} \tag{4.12}$$

$$\mathbf{r}^{(t)} = \sigma(\mathbf{W}_r[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(reset gate)} \tag{4.13}$$

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}[\mathbf{r}^{(t)} \odot \mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \tag{4.14}$$

$$\mathbf{h}^{(t)} = (1 - \mathbf{z}^{(t)}) \odot \mathbf{h}^{(t-1)} + \mathbf{z}^{(t)} \odot \tilde{\mathbf{h}}^{(t)} \tag{4.15}$$

Comparison with LSTM:

- Fewer parameters (2 gates vs 3)
- No separate cell state
- Often performs comparably to LSTM

4.7 Choosing Architecture

Table 4.1 – Architecture Selection Guide

Architecture	Best For	Examples
DNN	Tabular data	Classification, regression
CNN	Images, spatial data	Object detection, segmentation
RNN/LSTM	Sequences	NLP, time series
Transformer	Long sequences	Translation, generation

4.8 Chapter Summary

1. DNNs are fully-connected networks for general-purpose learning
2. CNNs exploit spatial structure through local connections
3. RNNs process sequential data with recurrent connections
4. LSTM and GRU address vanishing gradients with gating
5. Architecture choice depends on data type and task

Chapter 5

Core Operations in Neural Networks

5.1 Introduction

This chapter covers the fundamental building blocks of modern neural networks: convolution, pooling, and normalization operations. Understanding these operations is essential for designing and implementing effective neural network architectures.

5.2 Convolution Operation

Convolution is the cornerstone of convolutional neural networks (CNNs).

5.2.1 2D Convolution

For input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and kernel $\mathbf{K} \in \mathbb{R}^{K \times K}$:

$$y[i, j] = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} k[m, n] \cdot x[i + m, j + n] \tag{5.1}$$

In practice, we often add bias and use multiple kernels for multiple output channels.

[Convolution Parameters]

- **Kernel size:** $K \times K$ (common: 3×3, 5×5, 7×7)
- **Stride (s):** Step size for sliding window
- **Padding (p):** Zero padding around input
- **Dilation (d):** Spacing between kernel elements

Output size with padding and stride:

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - d(K - 1) - 1}{s} + 1 \right\rfloor \tag{5.2}$$

5.2.2 Grouped and Depthwise Convolution

- **Grouped convolution:** Split input channels into groups, apply separate conv per group
- **Depthwise convolution:** One kernel per input channel (grouped with $G = C_{in}$)
- **Pointwise convolution:** 1×1 conv (used to mix channels)

MobileNet uses: Depthwise Separable Convolution = Depthwise + Pointwise

[Depthwise Separable Convolution]

- **Standard conv FLOPs:** $H \cdot W \cdot K^2 \cdot C_{in} \cdot C_{out}$

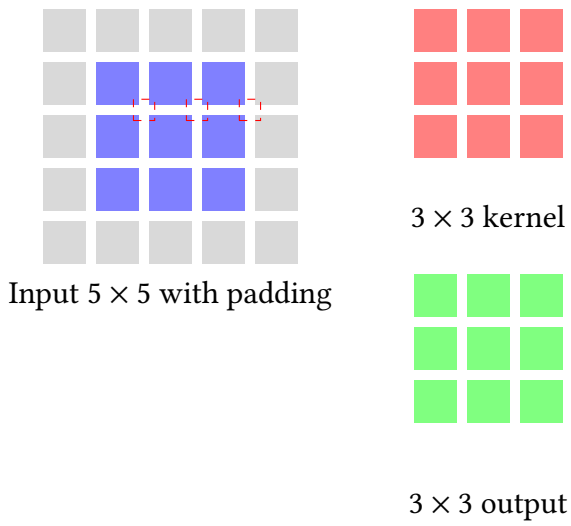


Figure 5.1 – Convolution with padding

- **Separable conv FLOPs:** $H \cdot W \cdot K^2 \cdot C_{in} + H \cdot W \cdot C_{in} \cdot C_{out}$
- **Speedup:** $\frac{1}{K^2} + \frac{1}{C_{out}}$ approximately

5.3 Pooling Operations

Pooling reduces spatial dimensions and provides translation invariance.

5.3.1 Max Pooling

$$y[i, j] = \max_{m,n \in \mathcal{N}(i,j)} x[m, n] \tag{5.3}$$

where $\mathcal{N}(i, j)$ is the neighborhood.

- **Retains strongest activation**
- **Provides local translation invariance**
- **Common:** 2×2 with stride 2

5.3.2 Average Pooling

$$y[i, j] = \frac{1}{K \cdot K} \sum_{m,n \in \mathcal{N}(i,j)} x[m, n] \tag{5.4}$$

- **Averages over neighborhood**
- **Less aggressive than max pooling**
- **Used in global pooling**

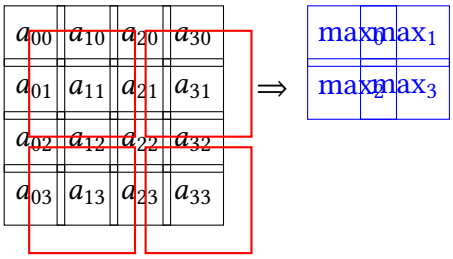


Figure 5.2 – Max pooling 2×2, stride 2

5.3.3 Global Pooling

Applied over entire feature map to produce single value per channel:

- **Global Average Pooling (GAP):** $\frac{1}{HW} \sum_{i,j} x_{ij}$
- **Global Max Pooling:** $\max_{i,j} x_{ij}$
- **Replaces FC layers** in modern architectures

5.4 Normalization Techniques

Normalization stabilizes training by controlling layer input distributions.

5.4.1 Batch Normalization

Normalizes activations across batch dimension:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \tag{5.5}$$

where:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} x_i \tag{5.6}$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (x_i - \mu_B)^2 \tag{5.7}$$

[BatchNorm Properties]

- **Reduces internal covariate shift**
- **Allows higher learning rates**
- **Provides slight regularization**
- **Dependent on batch size and can be problematic in small batches**

[BatchNorm vs. Inference] During training, BatchNorm uses batch statistics. During inference, use running statistics:

$$\hat{x} = \frac{x - \tilde{\mu}}{\sqrt{\tilde{\sigma}^2 + \epsilon}} \tag{5.8}$$

where $\tilde{\mu}$ and $\tilde{\sigma}^2$ are running averages.

5.4.2 Layer Normalization

Normalizes across features within a single sample:

$$\mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \quad \sigma = \frac{1}{D}\sum_{i=1}^{D}(x_i - \mu)^2 \tag{5.9}$$

- **Independent of batch size**
- **Used in RNNs and Transformers**
- **Same for training and inference**

Table 5.1 – Normalization Comparison

Method	Normalize over	Batch dep.	Use case
BatchNorm	(H, W, C)	Yes	CNNs
LayerNorm	(H, W, N)	No	Transformers
InstanceNorm	(H, W)	No	Style transfer
GroupNorm	(H, W, G)	No	Small batch CNNs

5.4.3 Instance Normalization

Normalizes each sample and channel independently:

$$y_{nchw} = \frac{x_{nchw} - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \epsilon}} \tag{5.10}$$

Used in style transfer where instance-specific statistics matter.

5.5 Combining Operations

Modern networks combine these operations in standardized blocks.

5.5.1 Basic Conv Block

A typical convolutional block:

1. Convolution
2. Batch Normalization
3. Activation (ReLU/GELU)

Common patterns:

- Conv-BN-ReLU (VGG style)
- BN-ReLU-Conv (ResNet style, pre-activation)

– Conv Block in PyTorch

python import torch.nn as nn

class ConvBlock(nn.Module): def $_{init}{}_{(self,in_ch,out_ch,kernel_size=3,stride=1):super().}{}_{init}{}_{()padding=kernel_size//2self.block=nn.Sequential(nn.Co}$

def forward(self, x): return self.block(x)

Pre-activation block (ResNet style) class PreActBlock(nn.Module): def

$_{init}{}_{(self,in_ch,out_ch,stride=1):super().}{}_{init}{}_{()self.block=nn.Sequential(nn.BatchNorm2d(in_ch),nn.ReLU(inplace=True),nn.Conv2d(in_ch,out_ch,3,stride,1,bias=False),nn.}$

def forward(self, x): return self.block(x) + self.shortcut(x)

5.6 Chapter Summary

1. Convolution extracts local features with shared weights
2. Pooling provides spatial reduction and invariance
3. Normalization stabilizes training dynamics
4. Modern architectures combine these in standardized blocks
5. Choice of operations depends on task and data characteristics

Chapter 6

Skip Connections and Attention Mechanisms

6.1 Skip Connections

Skip connections (shortcuts) bypass layers to enable gradient flow in deep networks.

6.1.1 Residual Connections (ResNet)

The residual learning framework:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x} \tag{6.1}$$

where $\mathcal{F}$ is the learned residual mapping.

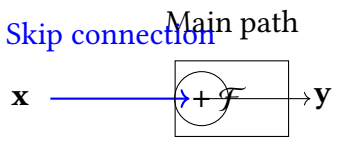


Figure 6.1 – Residual connection

If dimensions match: $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$
If dimensions differ: $\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}$
[ResNet Benefits]

- Enables training of very deep networks (100+ layers)
- Alleviates vanishing/exploding gradients
- Reduces degradation problem
- Easy to implement and computationally minimal

6.1.2 Bottleneck Architecture

Used in deeper ResNets to reduce complexity:

- 1×1 conv to reduce dimensions
- 3×3 conv (bottleneck)
- 1×1 conv to restore dimensions

For input C, bottleneck outputs C:

1. Conv $1 \times 1, C \longrightarrow C//4$
2. Conv $3 \times 3, C//4 \longrightarrow C//4$
3. Conv $1 \times 1, C//4 \longrightarrow C$

6.1.3 Dense Connections (DenseNet)

Each layer receives feature maps from all preceding layers:

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}]) \tag{6.2}$$

- **Feature reuse:** Reduces parameters
- **Strong gradient flow**
- **Higher memory cost** due to concatenation

6.2 Attention Mechanisms

Attention allows models to focus on relevant parts of the input.

6.2.1 Self-Attention

The attention function:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6.3}$$

where:

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$ (Query)
- $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ (Key)
- $\mathbf{V} = \mathbf{X}\mathbf{W}_V$ (Value)

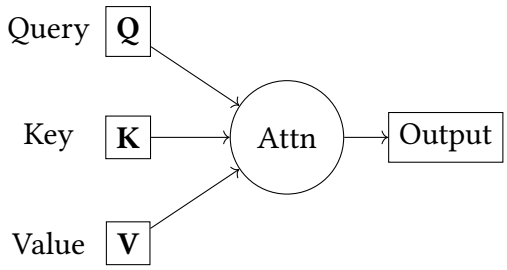


Figure 6.2 – Self-attention mechanism

The scaling factor $\sqrt{d_k}$ prevents vanishing gradients for large d_k.
[Self-Attention Properties]

- **Allows modeling long-range dependencies**
- **Computationally $O(n^2)$ but parallelizable**
- **Interpretable:** Attention weights show relevance

6.2.2 Squeeze-and-Excitation (SE) Attention

Channel attention mechanism:

1. **Squeeze:** Global average pooling $\mathbf{z} = \frac{1}{H \times W} \sum_{i,j} \mathbf{X}_{:,i,j}$
2. **Excitation:** FC-ReLU-FC-Sigmoid $\mathbf{s} = \sigma(W_2 \delta(W_1 \mathbf{z}))$
3. **Scale:** $\tilde{\mathbf{X}} = \mathbf{s} \cdot \mathbf{X}$

[SE Block]

- **Modeling channel dependencies**
- **Lightweight:** Minimal parameters
- **Plug-and-play:** Can be added to existing architectures

6.3 Chapter Summary

1. Skip connections enable deep networks by improving gradient flow
2. Residual connections are the most widely used skip connection
3. Attention mechanisms allow dynamic focus on relevant features
4. Self-attention is the foundation of Transformers
5. Channel attention (SE) provides efficient feature recalibration

Chapter 7

Optimizers and Learning Rate Schedulers

7.1 Gradient Descent Optimization

Training neural networks minimizes a loss function by updating parameters in the direction that reduces loss.

7.1.1 Batch Gradient Descent

Updates parameters using the gradient of the entire dataset:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t) \tag{7.1}$$

- **Advantages:** Stable convergence, exact gradient
- **Disadvantages:** Slow for large datasets, memory intensive

7.1.2 Stochastic Gradient Descent (SGD)

Updates parameters using gradient of a single sample:

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t; \mathbf{x}^{(i_t)}, \mathbf{y}^{(i_t)}) \tag{7.2}$$

- **Advantages:** Fast, escapes local minima, online learning
- **Disadvantages:** Noisy gradients, requires smaller learning rate

7.1.3 Mini-Batch Gradient Descent

Updates using a batch of B samples:

$$\theta_{t+1} = \theta_t - \alpha \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathcal{L}(\theta_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \tag{7.3}$$

This is the standard approach in deep learning.

7.2 Gradient Descent with Momentum

Momentum accelerates convergence by accumulating velocity:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \nabla_\theta \mathcal{L}(\theta_t) \tag{7.4}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_{t+1} \tag{7.5}$$

- $\beta \in [0, 1)$ is the momentum coefficient (typically 0.9)
- **Reduces oscillations** in steep directions
- **Accelerates convergence** in shallow directions

[Nesterov Accelerated Gradient] Nesterov momentum looks ahead:

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t - \alpha \nabla_\theta \mathcal{L}(\theta_t + \beta \mathbf{v}_t) \tag{7.6}$$
$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1} \tag{7.7}$$

This provides better convergence guarantees and often faster training.

7.3 Adaptive Learning Rate Methods

These methods adjust learning rates per-parameter based on gradient history.

7.3.1 AdaGrad

Adapts learning rate inversely proportional to accumulated squared gradients:

$$\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t) \tag{7.8}$$
$$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t \tag{7.9}$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \tag{7.10}$$

- **Good for sparse features**
- **Learning rate decreases** over time
- **May stop too early** in deep learning

7.3.2 RMSProp

Exponentially weighted moving average of squared gradients:

$$\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t) \tag{7.11}$$
$$\mathbf{r}_t = \beta \mathbf{r}_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \tag{7.12}$$
$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \tag{7.13}$$

- **Addresses AdaGrad's diminishing learning rate**
- **Common default for RNNs**
- $\beta = 0.9$ is typical

7.3.3 Adam (Adaptive Moment Estimation)

Combines momentum and RMSProp:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t \quad \text{(first moment)} \tag{7.14}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2 \quad \text{(second moment)} \tag{7.15}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \quad \text{(bias correction)} \tag{7.16}$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \tag{7.17}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{7.18}$$

[Adam Properties]

- **Default optimizer** for most deep learning tasks
- **Robust to hyperparameters**
- **Default parameters:** $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$
- **Bias correction** for early time steps

– Optimizers in PyTorch

python import torch.optim as optim
SGD with momentum optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
SGD with Nesterov momentum optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True)
Adam optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))
AdamW (decoupled weight decay) optimizer = optim.AdamW(model.parameters(), lr=0.001, weight$_decay = 0.01$)
RMSProp optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)

7.3.4 AdamW (Adam with Weight Decay)

Decouples weight decay from gradient update:

$$\theta_{t+1} = \theta_t - \alpha \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon} - \alpha \lambda \theta_t \tag{7.19}$$

- **Proper L2 regularization** unlike original Adam
- **Better generalization** in many tasks
- **Standard in transformers**

7.4 Gradient Clipping

Prevents exploding gradients:

$$\mathbf{g} = \text{clip}(\nabla_\theta \mathcal{L}, -\text{clip_value}, \text{clip_value}) \tag{7.20}$$

Common values: clip_value = 1.0 to 5.0

Essential for RNNs and Transformers.

7.5 Learning Rate Schedulers

Learning rate schedules adjust the learning rate during training.

7.5.1 Step Decay

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/T \rfloor} \tag{7.21}$$

where γ is the decay factor (e.g., 0.5) and T is the period.

7.5.2 Exponential Decay

$$\alpha_t = \alpha_0 \cdot e^{-\lambda t} \tag{7.22}$$

7.5.3 Cosine Annealing

$$\alpha_t = \alpha_{min} + \frac{1}{2}(\alpha_{max} - \alpha_{min}) \left(1 + \cos\left(\frac{t\pi}{T}\right)\right) \tag{7.23}$$

Figure 7.1 – Learning rate schedules

7.5.4 Warmup

Gradually increase learning rate at the start:

$$\alpha_t = \alpha_{max} \cdot \frac{t}{T_{warmup}} \quad \text{for } t < T_{warmup} \tag{7.24}$$

- **Prevents large gradients** at the start
- **Important for transformers**
- **Common: 500-10000 warmup steps**

7.6 Optimizer Selection

7.7 Chapter Summary

1. Gradient descent is the foundation of neural network optimization
2. Momentum accelerates convergence by accumulating velocity
3. Adam combines momentum and adaptive learning rates
4. AdamW provides proper weight decay regularization
5. Learning rate schedulers improve convergence

Table 7.1 – Optimizer Selection Guide

Optimizer	Good for	Notes
SGD + Momentum	Image classification	Best generalization
Adam	General purpose	Default choice
AdamW	Transformers, NLP	Best for fine-tuning
RMSProp	RNNs	Good for recurrent models

6. Warmup is important for transformer training

Chapter 8

Classic Network Architectures

8.1 Evolution of Neural Networks

The history of deep learning shows a progression from simple networks to increasingly complex architectures, each building upon innovations of the previous era.

8.2 Perceptron and Early Neural Networks

8.2.1 The Perceptron (1958)

The perceptron, introduced by Frank Rosenblatt, was the first attempt at artificial neural networks:

$$y = \begin{cases} 1 & \text{if } \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{otherwise} \end{cases} \tag{8.1}$$

Limitations: Can only solve linearly separable problems (XOR problem showed this).

8.3 LeNet (1998)

Yann LeCun's LeNet was the first successful CNN for digit recognition.

[LeNet-5 Architecture]

1. Input: 32×32 grayscale image
2. C1: 6 filters of 5×5, stride 1 $\longrightarrow 28 \times 28 \times 6$
3. S2: 2×2 avg pool, stride 2 $\longrightarrow 14 \times 14 \times 6$
4. C3: 16 filters of $5 \times 5 \longrightarrow 10 \times 10 \times 16$
5. S4: 2×2 avg pool $\longrightarrow 5 \times 5 \times 16$
6. C5: 120 filters of $5 \times 5 \longrightarrow 120$
7. F6: Fully connected, 84 units
8. Output: 10 classes (digits)

Total parameters: ~60,000

Key innovations:

- Weight sharing through convolutions
- Local receptive fields
- Spatial subsampling (pooling)

8.4 AlexNet (2012)

AlexNet won ImageNet LSVRC-2012 with a significant margin, revolutionizing computer vision.

[AlexNet Architecture]

1. Input: $224 \times 224 \times 3$ RGB image
2. Conv1: 96 filters of 11×11, stride 4 $\rightarrow 55 \times 55 \times 96$
3. Pool1: 3×3 max pool, stride 2 $\rightarrow 27 \times 27 \times 96$
4. Norm1: Response normalization
5. Conv2: 256 filters of $5 \times 5 \rightarrow 27 \times 27 \times 256$
6. Pool2: 3×3 max pool $\rightarrow 13 \times 13 \times 256$
7. Conv3: 384 filters of 3×3
8. Conv4: 384 filters of 3×3
9. Conv5: 256 filters of 3×3
10. Pool5: 3×3 max pool $\rightarrow 6 \times 6 \times 256$
11. FC6: 4096 neurons
12. FC7: 4096 neurons
13. FC8: 1000 classes (ImageNet)

Key innovations:

- ReLU activation (faster training)
- GPU training (5-6 days vs weeks)
- Data augmentation
- Dropout regularization
- Response normalization

Parameters: ~60 million

8.5 VGGNet (2014)

VGGNet emphasized depth and simplicity, using only 3×3 convolutions.
[VGG-16 Architecture]

1. Input: $224 \times 224 \times 3$
2. Conv blocks:
    - Block 1: 2× conv 64, pool
    - Block 2: 2× conv 128, pool
    - Block 3: 3× conv 256, pool
    - Block 4: 3× conv 512, pool
    - Block 5: 3× conv 512, pool
3. FC: 4096, 4096, 1000

Key insight: Two 3×3 convolutions have receptive field of 5×5 but fewer parameters:

- 5×5 conv: $25C^2$ parameters
- Two 3×3 convs: $18C^2$ parameters (28% reduction)
- More nonlinearities with two ReLUs

Parameters: ~138 million

8.6 GoogLeNet / Inception (2014)

Introduced the Inception module with parallel multi-scale convolutions.
[Inception Module] Parallel operations at different scales:

- 1×1 conv (bottleneck)
- 3×3 conv (after 1×1)
- 5×5 conv (after 1×1)
- 3×3 max pool (after 1×1)

Concatenate all outputs along channel dimension.
Key innovations:

- Multi-scale feature extraction
- Dimensionality reduction with 1×1 conv
- Global Average Pooling
- Auxiliary classifiers for training

Parameters: ~4 million (very efficient!)

8.7 ResNet (2015)

ResNet introduced residual connections, enabling training of very deep networks.
[ResNet-50 (Bottleneck)]

1. Conv1: 7×7, 64, stride 2
2. Pool: 3×3 max, stride 2
3. Stage 1: 3 blocks, 64 channels
4. Stage 2: 4 blocks, 128 channels
5. Stage 3: 6 blocks, 256 channels
6. Stage 4: 3 blocks, 512 channels
7. Pool: Global Average
8. FC: 1000 classes

Key innovations:

- Skip connections for gradient flow
- Identity shortcuts
- Bottleneck design (1×1, 3×3, 1×1)
- Enables 1000+ layers

Parameters: ~25 million

8.8 DenseNet (2017)

Dense connections where each layer receives inputs from all preceding layers.
[DenseNet Block] For a block with L layers, each receives $k_0 + k \times (l - 1)$ inputs:

$$\mathbf{x}_1 = H_1([\mathbf{x}_0]) \tag{8.2}$$
$$\mathbf{x}_2 = H_2([\mathbf{x}_0, \mathbf{x}_1]) \tag{8.3}$$
$$\mathbf{x}_3 = H_3([\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2]) \tag{8.4}$$
$$\vdots \tag{8.5}$$
$$\mathbf{x}_L = H_L([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{L-1}]) \tag{8.6}$$

Key innovations:

- Feature reuse reduces parameters
- Strong gradient flow
- Better parameter efficiency than ResNet

Parameters: ~27 million (similar to ResNet but better)

8.9 Transformer (2017)

The Transformer architecture replaced recurrence with self-attention.
[Transformer Architecture]

1. **Encoder:** N identical layers
    - Multi-head self-attention
    - Feed-forward network
    - Layer normalization
    - Residual connections
2. **Decoder:** N identical layers
    - Masked multi-head self-attention
    - Encoder-decoder attention
    - Feed-forward network
3. **Positional encoding** (sinusoidal or learned)

Key innovations:

- Self-attention replaces recurrence
- Parallelizable (unlike RNNs)
- Captures long-range dependencies
- Scalable with computation

Parameters: 65M (base) to 175B (GPT-3) and beyond

Table 8.1 – Architecture Comparison

Network	Year	Parameters	Top-1 Acc.
AlexNet	2012	60M	62.5%
VGG-16	2014	138M	71.5%
GoogLeNet	2014	4M	72.0%
ResNet-50	2015	25M	76.1%
DenseNet-121	2017	8M	74.4%
ViT-B/16	2021	86M	79.9%

8.10 Chapter Summary

1. LeNet pioneered CNNs for digit recognition
2. AlexNet popularized deep learning with GPU training
3. VGG showed depth through 3×3 convolutions
4. Inception introduced multi-scale processing
5. ResNet enabled ultra-deep networks with skip connections
6. Transformer revolutionized NLP and later vision with attention

Chapter 9

Parameter and Computation Analysis

9.1 Importance of Model Efficiency

Understanding model efficiency is crucial for:

- Deployment on resource-constrained devices
- Training cost estimation
- Model selection and architecture design
- Benchmarking and comparison

Key metrics:

- **Parameters:** Model size and memory footprint
- **FLOPs:** Computational complexity
- **Latency:** Inference time
- **Throughput:** Samples per second

9.2 Parameter Counting

Parameters are the learnable weights and biases in the network.

9.2.1 Fully Connected Layer

For input dimension d_{in} and output dimension d_{out}:

$$\text{Weights} = d_{in} \times d_{out} \tag{9.1}$$

$$\text{Biases} = d_{out} \tag{9.2}$$

$$\text{Total} = d_{in} \times d_{out} + d_{out} \tag{9.3}$$

Example: FC layer from 784 to 256:

$$784 \times 256 + 256 = 200,704 + 256 = 200,960 \text{ parameters} \tag{9.4}$$

9.2.2 Convolutional Layer

For input channels C_{in}, output channels C_{out}, kernel size K:

$$\text{Weights} = K \times K \times C_{in} \times C_{out} \tag{9.5}$$

$$\text{Biases} = C_{out} \quad \text{(typically)} \tag{9.6}$$

$$\text{Total} = K^2 \times C_{in} \times C_{out} + C_{out} \tag{9.7}$$

Example: Conv layer 3×3, 64 in, 128 out:

$$3 \times 3 \times 64 \times 128 + 128 = 73,728 + 128 = 73,856 \text{ parameters} \tag{9.8}$$

9.2.3 Batch Normalization

For C channels:

$$\gamma \text{ (scale)} = C \tag{9.9}$$
$$\beta \text{ (shift)} = C \tag{9.10}$$
$$\mu \text{ (running mean)} = C \quad \text{(inference only)} \tag{9.11}$$
$$\sigma \text{ (running var)} = C \quad \text{(inference only)} \tag{9.12}$$
$$\text{Total trainable} = 2C \tag{9.13}$$

Note: During training, BN has 2 trainable and 2 non-trainable parameters per channel.

9.3 FLOPs (Floating Point Operations)

FLOPs measure the computational cost of a forward pass.

9.3.1 FLOPs for FC Layer

For matrix multiplication $\mathbf{Y} = \mathbf{WX} + \mathbf{b}$:

$$\text{Multiplications} = d_{out} \times d_{in} \quad \text{(per sample)} \tag{9.14}$$
$$\text{Additions} = d_{out} \times (d_{in} - 1) \quad \text{(approximately } d_{out} \times d_{in}\text{)} \tag{9.15}$$
$$\text{FLOPs} \approx 2 \times d_{out} \times d_{in} \quad \text{(counting mult+add as 2 FLOPs)} \tag{9.16}$$

For batch of N: FLOPs $= 2 \times N \times d_{out} \times d_{in}$

9.3.2 FLOPs for Conv Layer

For output feature map $H_{out} \times W_{out}$:

$$\text{FLOPs per output pixel} = K^2 \times C_{in} \times C_{out} \quad \text{(mult-add pairs)} \times 2 \tag{9.17}$$
$$\text{Total FLOPs} = H_{out} \times W_{out} \times K^2 \times C_{in} \times C_{out} \times 2 \tag{9.18}$$

Example: 224×224 input, 3×3 conv, 64 in, 128 out:

$$H_{out} \times W_{out} = 224 \times 224 = 50,176 \tag{9.19}$$
$$\text{FLOPs} = 50,176 \times 3 \times 3 \times 64 \times 128 \times 2 \approx 1.17 \text{ GFLOPs} \tag{9.20}$$

[MACs vs FLOPs] Sometimes FLOPs are reported as Multiply-Accumulate (MAC) operations:

$$\text{MAC} = \sum_i a_i \cdot b_i \tag{9.21}$$

Each MAC typically counts as 1 operation (or 2 FLOPs if counting mult and add sepa-

rately).

1 MAC ≈ 2 FLOPs for basic arithmetic.

9.4 Practical Examples

9.4.1 LeNet-5 Parameter Count

Layer	Input	Weights	FLOPs
Conv1	$1 \times 32 \times 32$	150	$2.7M$
Conv2	$6 \times 14 \times 14$	$2,400$	$4.2M$
FC1	400	$240K$	$480K$
FC2	120	$48K$	$48K$
Total	-	~60K	~7M

9.4.2 AlexNet Parameter Count

Layer	Output	Params	FLOPs
Conv1	$96 \times 55 \times 55$	35K	105M
Conv2	$256 \times 27 \times 27$	614K	448M
Conv3	$384 \times 13 \times 13$	885K	149M
Conv4	$384 \times 13 \times 13$	1.3M	224M
Conv5	$256 \times 13 \times 13$	884K	149M
FC6	4096	37.7M	37.7M
FC7	4096	16.8M	16.8M
FC8	1000	4.1M	4.1M
Total	-	~60M	~1.1B

9.5 Transfer Learning and Efficiency

Transfer learning can dramatically reduce required data and training.

[Transfer Learning Benefits]

- Lower training cost (no)
- Can use smaller datasets
- Faster convergence
- Often better final performance

Common strategies:

1. **Freeze:** Keep pretrained weights fixed
2. **Fine-tune:** Adapt all/some layers with small learning rate
3. **Extract features:** Use pretrained as feature extractor

9.6 Chapter Summary

1. Parameter count determines model size and memory requirements
2. FLOPs measure computational complexity
3. Efficient architectures like MobileNet reduce both
4. Transfer learning leverages pretrained models efficiently
5. Understanding efficiency guides model selection

Chapter 10

Training Process and Frameworks

10.1 The Training Loop

Neural network training consists of iterative optimization over data.

[Training Loop]

1. Initialize model weights
2. For each epoch:
   (a) Shuffle training data
   (b) For each batch:
      i. Forward pass: compute predictions
      ii. Compute loss
      iii. Backward pass: compute gradients
      iv. Update weights using optimizer
   (c) Evaluate on validation set
   (d) Save checkpoint if improved
3. Return best model

10.2 Hyperparameters

Critical hyperparameters that affect training:

10.2.1 Learning Rate

The most important hyperparameter.

- **Too high:** Divergence, exploding loss
- **Too low:** Slow convergence, local minima
- **Typical range:** 10^{-5} to 0.1
- **Default for Adam:** 10^{-3}
- **Default for SGD:** 10^{-2}

Finding good learning rates:

- Learning rate range test (LR finder)
- Start with known good defaults
- Reduce by factor of 10 if loss diverges

10.2.2 Batch Size

Number of samples per training step.

- **Small batch:** Noisier gradients, better generalization
- **Large batch:** Smoother gradients, faster but needs LR scaling
- **Typical values:** 16, 32, 64, 128, 256
- **Large batch scaling:** $\alpha_{large} = \alpha_{small} \times \frac{B_{large}}{B_{small}}$

[Batch Size vs. Generalization] Research shows smaller batches often achieve better generalization, but the effect depends on other factors like learning rate and network architecture.

Table 10.1 – Hyperparameter Ranges

Hyperparamete	Typical Range	Notes
Learning Rate	10^{-5} to 0.1	Adam: 10^{-3}, SGD: 10^{-2}
Batch Size	16-512	Depends on GPU memory
Epochs	10-1000	Depends on dataset size
Weight Decay	10^{-6} to 10^{-2}	AdamW: 10^{-2} typical
Dropout	0.1-0.5	Depends on regularization need
Momentum	0.9-0.999	For SGD

10.3 Regularization

Techniques to prevent overfitting:

10.3.1 Dropout

Randomly zero out activations during training:

$$\mathbf{y} = \mathbf{W} \cdot (\mathbf{x} \odot \mathbf{m}) \tag{10.1}$$

where $\mathbf{m} \sim \text{Bernoulli}(p)$ is the mask.

During inference, no dropout (or use inverted dropout).

[Dropout Effects]

- Ensemble of sub-networks
- Reduces co-adaptation
- Increases training time slightly
- Typical dropout rate: 0.1-0.5

10.3.2 Weight Decay

L2 regularization:

$$\mathcal{L}_{reg} = \mathcal{L} + \frac{\lambda}{2}\|\mathbf{W}\|_F^2 \tag{10.2}$$

Equivalent to:

$$\mathbf{W} \leftarrow (1 - \alpha\lambda)\mathbf{W} - \alpha\nabla\mathcal{L} \tag{10.3}$$

For AdamW: Weight decay is applied independently of the adaptive learning rate.

10.4 Early Stopping

Stop training when validation performance stops improving:

1. Monitor validation loss
2. Save best model
3. Stop if no improvement for N epochs
4. Return best model

Typical patience: 10-20 epochs

10.5 Model Checkpointing

Save model state during training:

- **Architecture:** Model definition
- **Weights:** Learned parameters
- **Optimizer state:** Momentum, etc.
- **Epoch:** Current training progress
- **Best metric:** For model selection

10.6 Backpropagation

The algorithm for computing gradients through the computational graph. [Backpropagation]

1. Forward pass: compute all intermediate values
2. Compute loss $\mathcal{L}$
3. Backward pass:
   (a) $\frac{\partial\mathcal{L}}{\partial\mathbf{y}}$ at output
   (b) For each layer (reverse order):
      i. Compute $\frac{\partial\mathcal{L}}{\partial\mathbf{x}} = \frac{\partial\mathcal{L}}{\partial\mathbf{y}} \cdot \frac{\partial\mathbf{y}}{\partial\mathbf{x}}$
      ii. Compute $\frac{\partial\mathcal{L}}{\partial\mathbf{W}}$ for weight update

[Chain Rule] Backpropagation applies the chain rule repeatedly:

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \frac{\partial y}{\partial x} \tag{10.4}$$

For complex graphs, this is computed efficiently using automatic differentiation.

10.7 PyTorch Framework

PyTorch is the most popular framework for deep learning research.

– Training Loop in PyTorch

python import torch import torch.nn as nn import torch.optim as optim from torch.utils.data import DataLoader

Model model = nn.Sequential(nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.2), nn.Linear(256, 10))

Loss and optimizer criterion = nn.CrossEntropyLoss() optimizer = optim.Adam(model.parameters(), lr=0.001, weight$_decay = 0.01)$

Training loop device = torch.device('cuda' if torch.cuda.is$_available()else'cpu')model = model.to(device)$

for epoch in range(num$_epochs)$: $model.train()train_loss = 0.0forbatch_idx, (data, target)inenumerate(train_loader)$: $data, target = data.to(device), target.to(device)$

optimizer.zero$_grad()output = model(data.view(data.size(0), -1))loss = criterion(output, target)loss.backward()optimizer.step()$

train$_loss+ = loss.item()$

Validation model.eval() val$_loss = 0.0correct = 0withtorch.no_grad()$: $fordata, targetinval_loader : data, target = data.to(device), target.to(device)output = model(data.view(data.size(0), -1))val_loss+ = criterion(output, target).item()pred = output.argmax(dim = 1)correct+ = pred.eq(target).sum().item()$

print(f'Epoch epoch: Train Loss=train$_loss/len(train_loader) : .4f,' f'ValLoss = val_loss/len(val_loader) : .4f,'$
$100. * correct/len(val_loader.dataset) : .2f$

10.8 CUDA and GPU Training

PyTorch uses CUDA for GPU acceleration.

– CUDA Operations in PyTorch

python import torch

Check CUDA availability print(torch.cuda.is$_available())print(torch.cuda.device_count())print(torch.cuda.ge$

Move tensors to GPU device = torch.device('cuda') x = torch.randn(1000, 1000).to(device)

y = torch.randn(1000, 1000).to(device) z = x @ y Matrix multiplication on GPU

Move model to GPU model = MyModel().to(device)

DataLoader for efficient loading train$_loader = DataLoader(train_dataset, batch_size = 32, shuffle = True, num_workers = 4, Paralleldataloadingpin_memory = TrueFasterGPUtransfer)$

Mixed precision training scaler = torch.cuda.amp.GradScaler() with torch.cuda.amp.autocast(): output = model(data) loss = criterion(output, target)

scaler.scale(loss).backward() scaler.step(optimizer) scaler.update()

[CUDA Principles]

- GPUs excel at parallel operations (matrix mult, convolutions)
- Data transfer between CPU and GPU is expensive
- Batching improves GPU utilization
- Mixed precision (FP16) reduces memory and increases speed

10.9 Chapter Summary

1. Training involves iterative optimization over batches
2. Hyperparameters (LR, batch size) critically affect training
3. Regularization prevents overfitting
4. Backpropagation computes gradients efficiently
5. PyTorch provides flexible deep learning primitives
6. CUDA enables GPU acceleration for training

Chapter 11

Dataset Processing

11.1 Data Collection

The foundation of any machine learning system is quality data.

11.1.1 Data Sources

- **Public datasets:** ImageNet, COCO, MNIST, etc.
- **Web scraping:** Automated data collection from websites
- **Sensors:** Cameras, microphones, medical devices
- **Human annotation:** Expert labeling
- **Synthetic data:** Generated from simulations

[Data Quality] Garbage in, garbage out. High-quality data is more important than sophisticated algorithms.

11.2 Data Annotation

Labels enable supervised learning.

11.2.1 Annotation Types

- **Classification:** Single label per image (cat vs dog)
- **Multi-label:** Multiple labels per image
- **Object detection:** Bounding boxes with labels
- **Semantic segmentation:** Pixel-wise labels
- **Instance segmentation:** Instance IDs per object
- **Keypoints:** Points of interest (faces, poses)

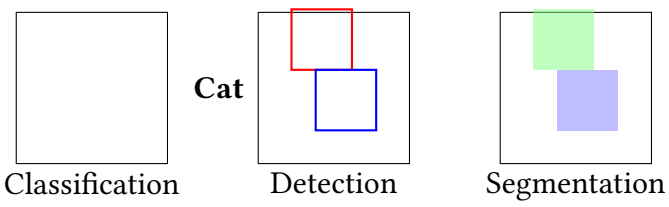


Figure 11.1 – Annotation types

11.2.2 Annotation Tools

- **LabelImg:** Object detection bounding boxes

- **CVAT:** Computer Vision Annotation Tool
- **Supervisely:** Full-featured annotation platform
- **Amazon Mechanical Turk:** Crowdsourced annotation
- **Label Studio:** Open-source annotation tool

11.3 Data Splitting

Proper data splitting is crucial for unbiased evaluation.

11.3.1 Basic Splits

- **Training set:** Used for model training
- **Validation set:** Used for hyperparameter tuning
- **Test set:** Used for final evaluation (only once)

Typical ratios:

- Small datasets (<10K): 70/10/20 or 80/10/10
- Large datasets (>1M): 98/1/1 may suffice

[Test Set Leakage] Never use test set for any decisions during training. This leads to overfitting to the test set and unreliable performance estimates.

11.3.2 Stratified Splitting

Maintains class distribution in each split:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y
)
```

11.3.3 Cross-Validation

For robust evaluation with limited data:

[K-Fold Cross-Validation]

1. Split data into K folds
2. For each fold:
   (a) Train on K-1 folds
   (b) Validate on remaining fold
3. Average performance across folds

Common values: K = 5 or 10

11.4 Data Augmentation

Artificially increase dataset size by transforming existing data.

11.4.1 Image Augmentations

- **Rotation:** Random angle rotation
- **Flip:** Horizontal/vertical flips
- **Crop:** Random crops
- **Color jitter:** Brightness, contrast, saturation
- **Scale:** Random scaling
- **Translation:** Random shifts
- **Noise:** Gaussian noise injection
- **Cutout:** Random rectangular masking
- **Mixup:** Blend two images and labels
- **CutMix:** Cut and paste patches between images

– Data Augmentation in PyTorch

python import torchvision.transforms as T
train$_transform = T.Compose([T.RandomResizedCrop(224), T.RandomHorizontalFlip(p = 0.5), T.RandomRotation(15), T.ColorJitter(brightness = 0.2, contrast = 0.2, saturation = 0.2, hue = 0.1), T.ToTensor(), T.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225])])$
val$_transform = T.Compose([T.Resize(256), T.CenterCrop(224), T.ToTensor(), T.Normalize(mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225])])$

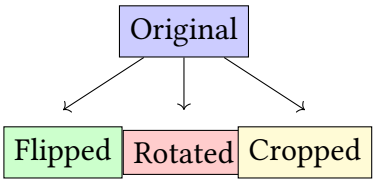


Figure 11.2 – Data augmentation examples

11.4.2 Advanced Augmentation

- **AutoAugment:** Learned augmentation policies
- **RandAugment:** Simpler policy search
- **TrivialAugment Wide:** No search needed
- **Test-Time Augmentation (TTA):** Augment at test time, average predictions

11.5 Data Loading

Efficient data loading is crucial for training speed.

– Data Loading in PyTorch

python from torch.utils.data import DataLoader, Dataset
class MyDataset(Dataset): def $_{init}{}_{(self,data,labels,transform=None):self.data=dataself.labels=labelsself.transform=transform}$
def $_{len}{}_{(self):returnlen(self.data)}$
def $_{getitem}{}_{(self,idx):image=self.data[idx]label=self.labels[idx]}$
if self.transform: image = self.transform(image)
return image, label
Create dataloaders train$_{loader} = DataLoader(train_{dataset}, batch_{size} = 32, shuffle = True, ImportantforSGDnum_{workers} = 4, Paralleldataloadingpin_{memory} = True, FasterGPUtransferdrop_{last} = TrueConsistentbatchsizes)$
Iterate for batch$_{idx}, (images, labels)inenumerate(train_{loader}) : images : (B, C, H, W)labels : (B,)pass$

[DataLoader Best Practices]

- Use multiple workers for parallel loading
- Enable pin_memory for faster GPU transfer
- Set shuffle=True for training
- Consider drop_last for consistent batch sizes

11.6 Normalization

Standardize input features for better training.

11.6.1 Image Normalization

$$x_{norm} = \frac{x - \mu}{\sigma} \tag{11.1}$$

Common ImageNet normalization:

- Mean: $[0.485, 0.456, 0.406]$
- Std: $[0.229, 0.224, 0.225]$

Compute dataset-specific normalization: python Compute mean and std loader = DataLoader(dataset, batch$_{size} = 32)mean = 0.0std = 0.0forimages,_{i}$ $nloader : batch_{samples} = images.size(0)images = images.view(batch_{samples}, images.size(1), -1)mean+ = images.mean(2).sum(0)std$ $images.std(2).sum(0)mean/= len(loader.dataset)std/= len(loader.dataset)$

11.7 Chapter Summary

1. Data quality is fundamental to model performance
2. Annotation types depend on the task
3. Proper data splitting prevents overfitting
4. Data augmentation improves generalization
5. Efficient data loading accelerates training
6. Normalization stabilizes training

Bibliography

[1] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.